

并行程序设计实验报告

CPU 架构编程

Cache 局部性、超标量与循环展开优化实验

姓名： 匡航逸 学号： 2412235
专业： 计算机科学与技术 日期： 2026 年 4 月 1 日
GitHub: <https://github.com/xzxxntxdy/Parallel-programming-NKU>

摘要

本文围绕两个典型 CPU 内核展开实验：矩阵列内积问题中的 cache 局部性优化，以及浮点数组求和问题中的超标量友好优化。按照作业要求，实验在 x86 平台上完成实现、基准测试与 VTune Hotspots 分析。本文实现了 `naive`、`cache_opt`、`superscalar` 和 `unroll` 等版本，并在不替代已有实现的前提下增加了额外的 `extreme` 版本，用于进一步考察分块转置、多累加器与更激进热路径组织的效果。

实验结果表明，矩阵列内积问题的主要瓶颈来自错误的数据访问模式。对 row-major 矩阵直接按列访问会带来严重跨步访存，而通过转置、连续访问和展开可以显著降低访存代价。在代表性大规模案例中，`RelWithDebInfo` 下 `matvec` 的 `extreme` 版本将平均运行时间从 `naive` 的 28.302ms 降至 1.665ms，达到 **17.00×** 加速。对 `sum` 而言，多累加器减少了 loop-carried dependency chain，在 `n=100000000` 时，`extreme` 将平均运行时间从 60.486ms 降至 40.535ms，达到 **1.49×** 加速。VTune Hotspots 结果与基准测试一致：优化版本的热点样本更加集中于预期核心 kernel，说明性能提升确实发生在热路径上。整体来看，本实验较完整地验证了 cache 优化、超标量友好设计、循环展开与编译器优化等级对 CPU 内核性能的共同影响。

关键词： cache 局部性，超标量，循环展开，VTune，benchmark

1 实验任务

本次作业要求围绕两个问题完成实现、基准测试、profiling 和分析：

1. 给定 $n \times n$ 矩阵 B 与向量 a ，计算每一列与向量的内积

$$s_j = \sum_{i=0}^{n-1} B_{i,j} a_i, \quad 0 \leq j < n.$$

要求至少提供按列访问的 `naive` 实现，以及一种明确改善 cache 局部性的优化版本。

2. 给定长度为 n 的浮点数组 x ，计算

$$S = \sum_{i=0}^{n-1} x_i.$$

要求至少提供单链式累加的 `naive` 实现，以及一种能够减少依赖链、体现 superscalar 思想的优化版本。

除必做部分外，作业还要求支持循环展开、不同编译优化等级比较、VTune/perf profiling、汇编分析，以及对浮点运算次序引起结果差异的说明。

2 实验平台与工程组织

2.1 实验平台

本实验在 **x86 平台** 上完成，使用 CMake 进行构建，并采用 VTune Hotspots 对代表性 workload 做 profiling。

表 2.1 实验平台与工程信息

项目	说明
实验平台	x86 平台
构建系统	CMake
构建模式	Debug、RelWithDebInfo、Release
profiling 工具	VTune Hotspots
推荐分析程序	build/<mode>/bench_cpu_arch
图表生成脚本	scripts/generate_report_assets.py
图表资源目录	report/figures

2.2 工程结构

整个项目按照作业要求组织为完整工程，核心目录包括：

- `src/`: 各算法实现与 benchmark 主程序；
- `include/`: 公共头文件与接口声明；
- `scripts/`: 构建、测试、批量 benchmark、VTune/perf 示例与报告资源生成脚本；
- `build/`: 不同构建模式的输出目录；
- `report/`: 报告正文、图表与数据摘要。

在算法层面，原有的 `naive`、`cache_opt`、`superscalar` 与 `unroll` 实现全部保留；在此基础上新增 `extreme` 路径，用于继续探索更激进但仍相对可维护的优化方案。

3 算法设计与实现

3.1 设计原则

本实验的两个任务虽然都属于“顺序读入一批数据并做归约”，但瓶颈并不相同：`matvec` 的主要问题是矩阵访问模式错误导致 **cache 利用率低**，而 `sum` 的主要问题是**单累加器形成长依赖链**，限制 **superscalar 处理器发掘 ILP**。因此，本文在实现时遵循两条原则：

1. 对 `matvec`，先修复数据布局和访存顺序，再讨论循环展开与更激进内核；
2. 对 `sum`，始终保持顺序访存，把优化重点放在减少依赖链、增加独立累加路径和降低循环控制开销上。

此外，benchmark 驱动将“准备阶段”和“计时阶段”明确分离：对于 `matvec` 的转置型算法，`prep_ms` 单独记录转置耗时，而 `total_ms/avg_ms/min_ms` 只统计真正的 kernel 执行时间。这样可以同时分析稳态性能与总成本，不会把一次性预处理和重复执行混为一谈。

3.2 Matvec：围绕数据布局的四种实现

设矩阵按 C 语言默认的 row-major 方式存放，原问题需要计算每一列与向量的点积：

$$s_j = \sum_{i=0}^{n-1} B_{i,j} a_i.$$

如果直接在原矩阵上按列读取，那么相邻两次访问地址之差为 n ，这正是 `matvecnaive` 性能差的根源。

3.2.1 naive：直接按列访问原矩阵

`naive` 版本直接实现数学定义。外层固定列 j ，内层遍历行 i ，在代码中访问 `matrix[row*n+col]`。它的特点是：

- 算法逻辑最清晰，没有额外预处理；

- 向量 `vector[row]` 是顺序访问的；
- 但矩阵元素是按 n 为步长跨行访问，空间局部性极差；
- 对大矩阵而言，一条 cache line 往往只用到很少几个元素，访存代价成为主导瓶颈。

换句话说，naive 的计算量并不比优化版本更多，真正拖慢它的是**访存模式与物理存储布局不匹配**。

3.2.2 cache_opt：先转置，再顺序点积

cache_opt 的思路是把“按列点积”改写成“对转置后矩阵的行做顺序点积”。在 benchmark 驱动中，当算法选择为 cache_opt 时，会先分配 `matrix_t`，然后调用 `transpose_f64` 或 `transpose_f32` 完成一次完整转置，并把这部分时间记入 `prep_ms`。真正进入计时区间后，kernel 只做：

1. 取出 `matrix_t+col*n` 作为当前“列”对应的连续内存段；
2. 对这段连续数据和向量做普通顺序点积；
3. 将结果写回 `out[col]`。

这个版本的关键收益在于：内层循环同时顺序读取 `column[row]` 和 `vector[row]`，硬件预取与缓存行利用率都显著改善。它 also 对应本实验最核心的结论之一：**对 matvec 来说，最重要的不是多条指令级小技巧，而是先把数据布局修正到适合 cache 的形式。**

3.2.3 unroll：在连续访存基础上减少循环开销

unroll 建立在 cache_opt 的正确数据布局之上。代码中使用宏生成了 `unroll=2/4/8` 三种版本，benchmark 默认使用 `unroll=4`。其核心结构是：

- 每次循环处理 FACTOR 个元素；
- 使用 `acc[lane]` 保存各 lane 的部分和；
- 主循环结束后先规约 `acc[]`，再处理尾部剩余元素。

这一设计有两个直接好处：

1. 减少了分支、计数器更新和边界判断的频率；
2. 把原本单一的累加链拆成多个相互独立的部分和，便于编译器做寄存器分配和指令调度。

因此，unroll 的收益来源不是“再次修复 cache”，而是在 cache 已经友好的前提下，进一步压缩热循环中的控制开销，并增加可并行执行的乘加对。

3.2.4 extreme：分块转置 + 更宽的计算内核

extreme 并不替代前面版本，而是作为额外的激进实现。它由两部分组成：

(1) 分块转置阶段 benchmark 驱动在选择 extreme 时，不再调用普通转置，而是调用 `transpose_blocked_f64 / transpose_blocked_f32`。该实现采用固定 `32x32` block：先枚举块左上角，再在块内部完成转置。这样做的目的不是改变最终数据布局，而是让**转置阶段本身**也更符合 cache 工作方式，减少大矩阵转置时的缓存抖动。

(2) 更激进的点积内核 在 f64 路径上，如果编译环境支持 SSE2，则 `matvec_extreme_f64` 使用四个 `__m128d` 累加器 `acc0..acc3`，每次主循环处理 8 个 double：

- 通过 `_mm_loadu_pd` 连续装载两元素；
- 使用 `_mm_mul_pd` 做向量乘法；
- 把结果分别累加到四条独立累加链中；
- 主循环结束后做 horizontal add，再处理剩余 2 个和 1 个元素的尾部。

若目标环境不支持 SSE/SSE2，则代码退回到标量版，但仍保留 8 个标量累加器，以维持较强的 ILP。与此同时，benchmark 驱动中所有大数组都通过 `aligned_malloc_or_die(64,...)` 分配为 64 字节对齐内存，并在实现中结合 `RESTRICT` 与 `ASSUME_ALIGNED` 提示编译器更大胆地优化别名分析和访存指令选择。

因此, **extreme** 的性能优势来自三个层次的叠加:

1. 分块转置降低准备阶段缓存扰动;
2. 连续布局保证稳态内层循环仍然是 cache-friendly 的;
3. 更多独立累加器和更宽的 SIMD-friendly 循环结构降低了每次点积的控制和规约开销。

3.3 Sum: 围绕依赖链的四种实现

sum 问题没有矩阵那样复杂的数据布局问题, 因为数组天然就是顺序存放、顺序读取的。它的主要瓶颈是: 如果始终写成 `sum+=data[i]`, 那么每次加法都依赖前一次结果, 处理器虽然具备 **superscalar** 发射能力, 也难以把一长串相关加法真正并行起来。

3.3.1 naive: 单累加器基线

naive 版本只有一个累加器 `sum`, 每轮循环执行一次读和一次加法:

$$s_{i+1} = s_i + x_i.$$

其优点是简单、开销低、代码最短; 缺点是整个循环几乎只有一条很长的归约依赖链。对于现代 **superscalar** CPU, 这通常意味着浮点加法吞吐能力利用不充分。

3.3.2 superscalar: 四路独立累加器

superscalar 版本显式维护 `acc0..acc3` 四个累加器, 每次循环处理 4 个元素。这样原来的一条长链被拆成了四条较短链:

$$acc_k \leftarrow acc_k + x_{i+k}, \quad k \in \{0, 1, 2, 3\}.$$

主循环结束后, 再执行一次

$$(acc_0 + acc_1) + (acc_2 + acc_3)$$

完成最终规约, 尾部不足 4 个的元素则继续并入 `acc0`。这种写法的核心价值是让处理器更容易同时发射和重叠多个加法, 从而提高 ILP。

3.3.3 unroll: 参数化展开与多路部分和

unroll 与 **matvec** 类似, 也通过宏生成了 2/4/8 三档版本。它的结构是:

- 定义 `acc[FACTOR]` 作为多路部分和;
- 每次循环处理 `FACTOR` 个输入;
- 主循环结束后先对 `acc[]` 做一次规约, 再处理尾元素。

和 **superscalar** 相比, **unroll** 的主要差别在于它是**参数化的展开框架**: 不仅减少了循环控制次数, 也允许我们在相同代码结构下直接比较不同 **unroll** 因子对性能的影响。报告中的默认结果使用 **unroll=4**, 是可读性、代码规模和效果之间较平衡的选择。

3.3.4 extreme: 面向规模的双路径求和内核

extreme 是 **sum** 中最有意思的实现, 因为它不是简单地“再多加几个累加器”, 而是显式考虑输入规模。代码中定义了阈值

$$\text{SUM_EXTREME_STREAMING_THRESHOLD} = 50000000.$$

当 n 不小于该阈值时, 程序进入 `streaming_impl` 路径; 否则优先使用 SSE/SSE2 路径。

(1) **大规模 streaming 路径** `sum_extreme_f64_streaming_impl` 维护 8 个标量累加器 `acc0..acc7`。主循环一次处理 16 个元素，并把 `i` 和 `i+8` 两组元素配对分摊到 8 条累加链上。这样做有三个目的：

1. 进一步拉长每个主循环体，减少循环控制比例；
2. 把一条很长的依赖链拆成 8 条较短链；
3. 在超大数组流式访问时保持简单、稳定、开销低的规约模式。

(2) **中等规模 SSE/SSE2 路径** 当问题规模还没有大到完全被内存带宽主导时，`sum_extreme_f64` 会优先走 SSE2 分支。该分支使用 4 个 `__m128d` 累加器，每次加载 8 个 `double`，并在循环末尾做 horizontal add。这个版本更像“显式向量寄存器上的多累加器求和”，对于中等规模输入通常有较好的吞吐表现。

也就是说，`extreme` 的设计思想不是单纯追求更复杂，而是根据 workload 的规模选择更合适的归约组织方式：**中等规模利用向量寄存器吞吐，大规模则采用更稳定的流式多累加器结构。**

3.4 正确性校验与浮点误差

由于不同实现会改变加法顺序，而浮点加法不满足结合律，因此实验不能简单要求逐位相等。本项目采用确定性输入生成方式，并以 `longdouble` 参考实现作为校验基准，同时检查绝对误差与相对误差：

$$\text{abs_err} = |y - y^{(\text{ref})}|, \quad \text{rel_err} = \frac{|y - y^{(\text{ref})}|}{|y^{(\text{ref})}| + \varepsilon}.$$

在当前实验结果中，所有构建模式下的正确性测试均通过：

- Debug: 通过；
- RelWithDebInfo: 通过；
- Release: 通过。

4 实验方法

4.1 统一 benchmark 入口

工程提供统一可执行程序 `bench_cpu_arch`，通过命令行参数切换任务、算法和规模，例如：

Listing 1 统一 benchmark 入口示例

```
1 ./build/RelWithDebInfo/bench_cpu_arch \
2   --task matvec --algo extreme --n 2048 --repeat 50 \
3   --warmup 3 --dtype f64 --build-mode RelWithDebInfo
4
5 ./build/RelWithDebInfo/bench_cpu_arch \
6   --task sum --algo extreme --n 100000000 --repeat 30 \
7   --warmup 3 --dtype f64 --build-mode RelWithDebInfo
```

程序输出统一包含 `check`、`total_ms`、`avg_ms`、`min_ms` 等字段，便于后续批量整理、对比不同构建模式，并直接导出到报告图表。

4.2 计时指标

本实验使用高精度计时统计总时长、平均时长和最小时长，并采用

$$\text{Speedup} = \frac{T_{\text{naive}}}{T_{\text{opt}}}$$

来量化相对 `naive` 的加速比。对含有一次性准备阶段的转置型 `matvec` 实现，还额外定义摊销成本：

$$T_{\text{amortized}} = T_{\text{kernel}} + \frac{T_{\text{setup}}}{R},$$

其中 R 是重复次数。该指标能更公平地比较“准备开销 + 稳态计算”共同作用下的整体收益。

4.3 规模设计

为避免只在单个点上做结论，工程同时包含两类测试：

- **批量 benchmark**：扫描多组规模，用于观察趋势；
- **代表性大规模案例**：即真正进入 VTune 的 workload，用于评价优化在真实 profiling 场景中的表现。

本报告重点使用如下代表性输入：

- **matvec**: $n = 2048$, repeat=50, warmup=3;
- **sum**: $n = 100000000$, repeat=30, warmup=3。

批量 benchmark 中最大的展示规模为：

- **matvec**: $n = 1536$;
- **sum**: $n = 20000000$ 。

5 批量 Benchmark 结果与分析

5.1 RelWithDebInfo 下的规模趋势

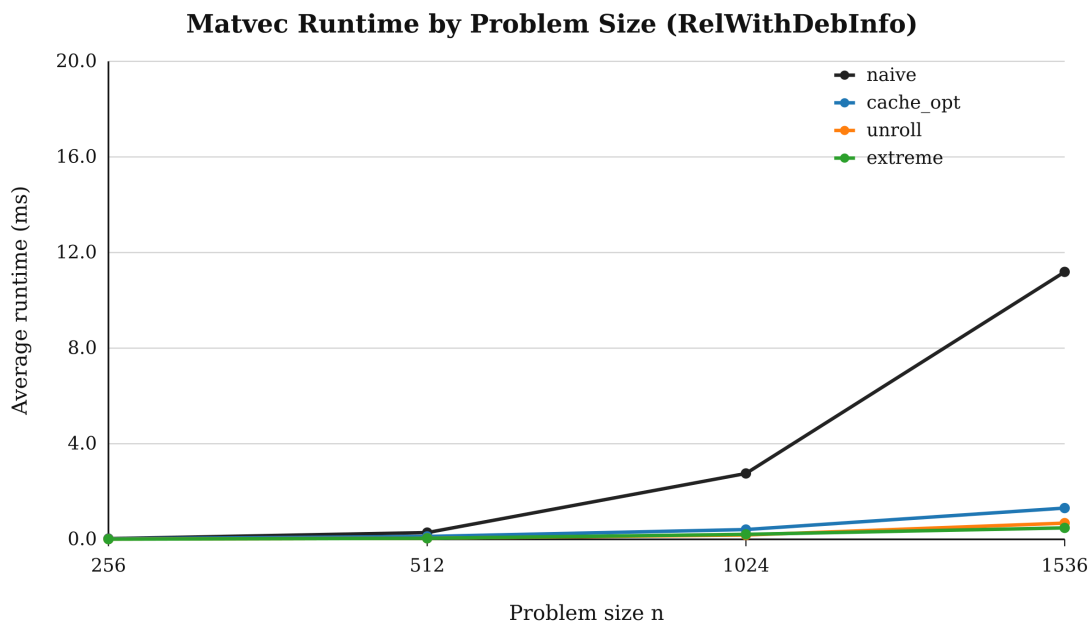


图 5.1 RelWithDebInfo 下 matvec 平均运行时间随规模变化趋势

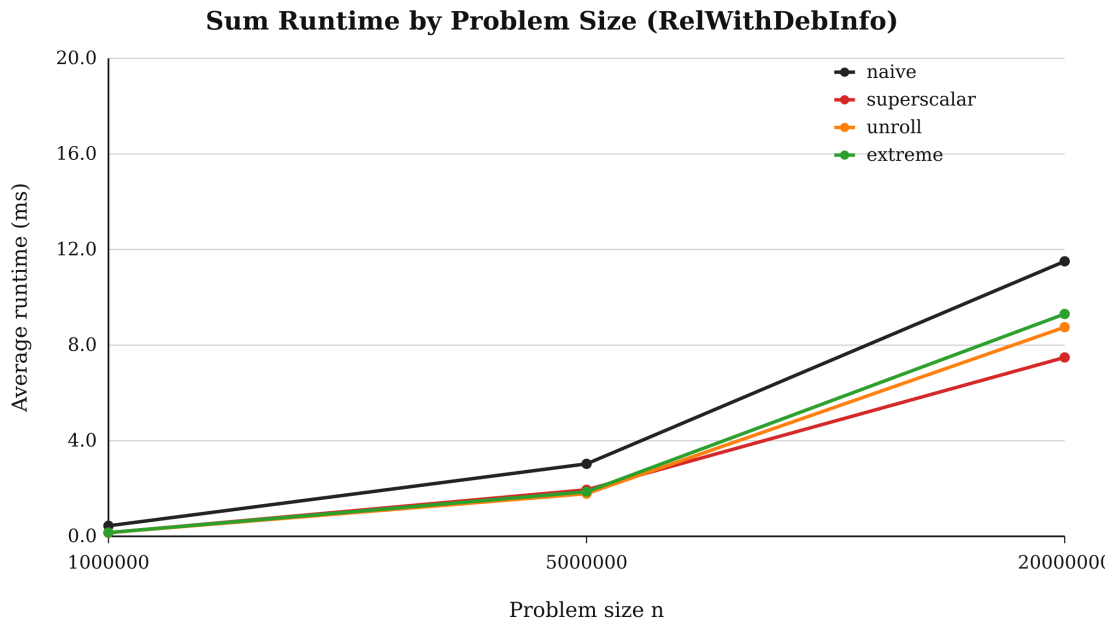


图 5.2 RelWithDebInfo 下 sum 平均运行时间随规模变化趋势

图 5.1 非常清楚地展示了矩阵列内积的核心现象：**naive** 由于始终按列跨步访问 row-major 矩阵，随着 n 增大，运行时间迅速恶化；而 **cache_opt**、**unroll** 与 **extreme** 都把热路径改造成连续访存，因此在大规模下和基线拉开数量级差距。在最大 batch case $n = 1536$ 处，**naive** 平均时间为 11.190 ms，而 **extreme** 仅为 0.479 ms，达到 **23.35 $\times$** 加速。

图 5.2 则体现了另一类行为：求和问题虽然也能通过多累加器和展开获得明显收益，但当规模变大后，它逐渐接近流式读内存场景，瓶颈更容易被带宽主导，因此不同优化版本之间的差距显著小于 **matvec**。这说明对 **sum**，**ILP 优化是必要的**，但其收益上界通常低于“修复错误访问模式”的 **cache 优化**。

5.2 最大 batch case 的定量结果

表 5.1 RelWithDebInfo 下最大 batch case 的结果

任务	算法	Avg ms	Min ms	Check
matvec, $n = 1536$	cache_opt	1.308	1.210	OK
	extreme	0.479	0.364	OK
	naive	11.190	9.303	OK
	unroll	0.679	0.452	OK
sum, $n = 20000000$	extreme	9.304	6.987	OK
	naive	11.507	9.817	OK
	superscalar	7.486	6.802	OK
	unroll	8.752	6.808	OK

表 5.1 说明，对 **matvec** 而言，仅仅通过转置修复访问模式就已经带来 $\frac{11.190384}{1.308325} \approx 8.55\times$ 的加速，这说明**数据布局修复本身就是主导收益来源**；继续做循环展开和更激进内核组织后，加速比分别提高到 **16.48 $\times$** 与 **23.35 $\times$** 。相比之下，**sum** 的改进更温和：在 $n = 20000000$ 这一批量规模上，**superscalar** 反而优于 **extreme**，说明这一问题更容易在不同输入区间下表现出不同的最优实现。

5.3 不同构建模式下的加速比

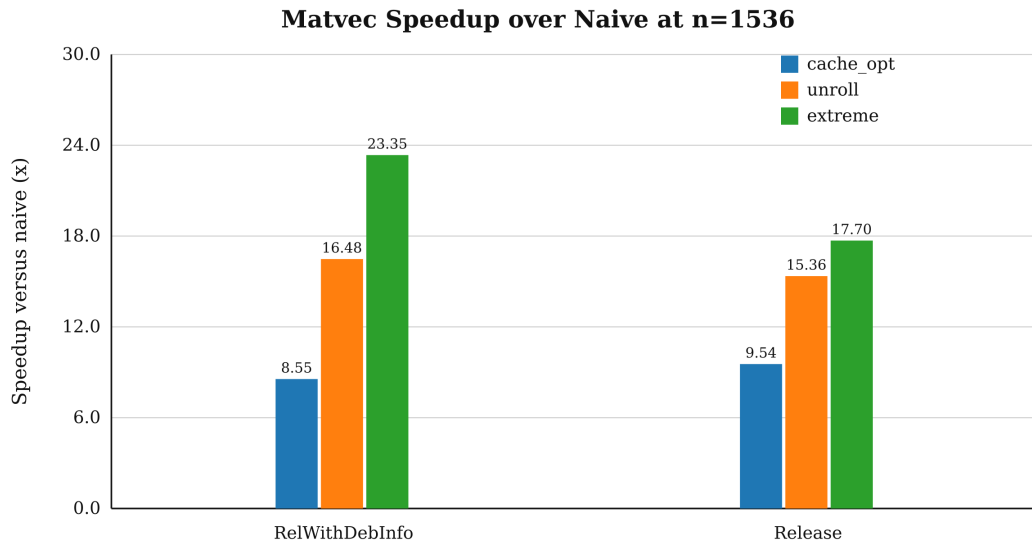


图 5.3 matvec 在不同构建模式下相对 naive 的加速比

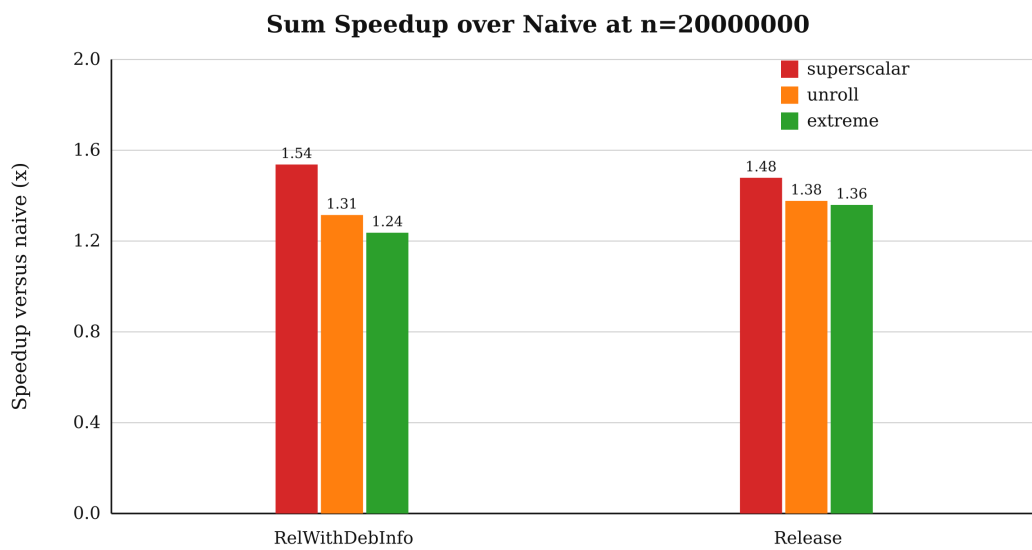


图 5.4 sum 在不同构建模式下相对 naive 的加速比

表 5.2 最大 batch case 上相对 naive 的加速比: RelWithDebInfo 与 Release 对比

任务	算法	RelWithDebInfo	Release
matvec, $n = 1536$	cache_opt	8.55×	9.54×
	unroll	16.48×	15.36×
	extreme	23.35×	17.70×
sum, $n = 20000000$	superscalar	1.54×	1.48×
	unroll	1.31×	1.38×
	extreme	1.24×	1.36×

图 5.3、图 5.4 和表 5.2 共同说明：编译器优化等级确实重要，但它并不能替代算法层面的结构性改动。对 `matvec` 来说，无论是 `RelWithDebInfo` 还是 `Release`，最关键的都是让热路径从跨步访存变为连续访存；对 `sum` 来说，编译器优化与多累加器策略的收益耦合得更紧密，但整体仍保持“优化版稳定优于基线”的趋势。

6 代表性大规模案例分析

批量 benchmark 用于观察趋势，但真正进入 VTune 的 workload 更大，因此需要单独分析，避免出现“图上看起来更快，但换到 profiling 规模后行为改变”的情况。

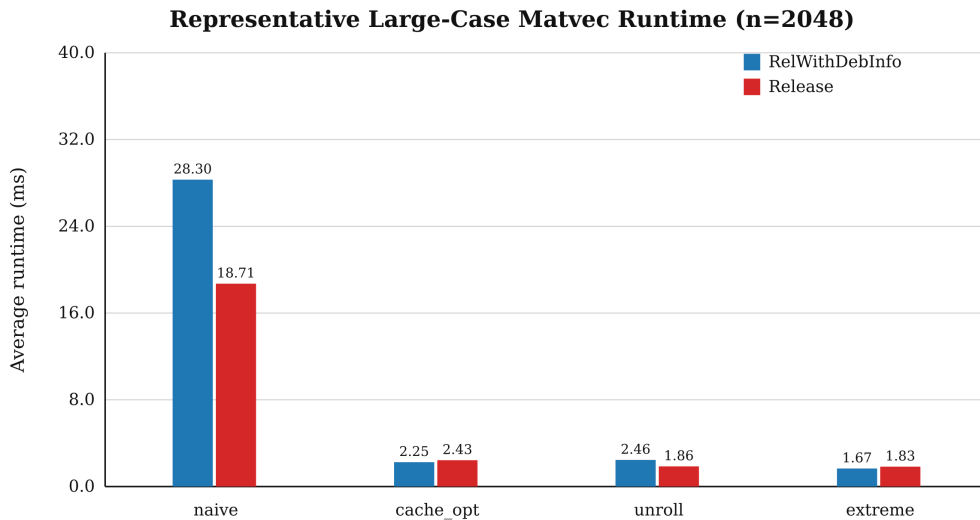


图 6.1 代表性大规模 `matvec` 案例比较： $n = 2048$

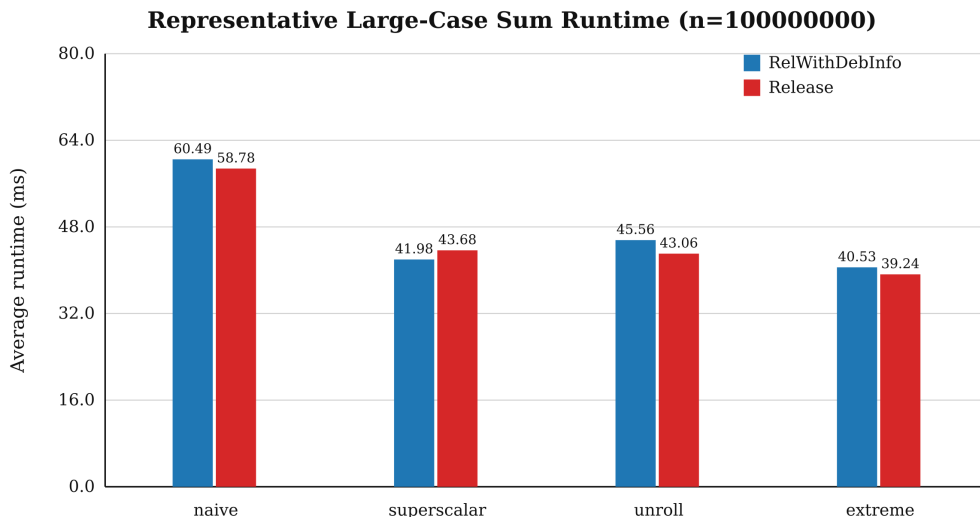


图 6.2 代表性大规模 `sum` 案例比较： $n = 100000000$

表 6.1 代表性大规模案例：RelWithDebInfo

任务	算法	n	Avg ms	Min ms	Check
matvec	cache_opt	2048	2.248	2.093	OK
	extreme	2048	1.665	1.494	OK
	naive	2048	28.302	22.682	OK
	unroll	2048	2.455	1.874	OK
sum	extreme	100000000	40.535	36.755	OK
	naive	100000000	60.486	56.892	OK
	superscalar	100000000	41.983	39.851	OK
	unroll	100000000	45.561	41.923	OK

表 6.2 代表性大规模案例：Release

任务	算法	n	Avg ms	Min ms	Check
matvec	cache_opt	2048	2.431	2.284	OK
	extreme	2048	1.835	1.612	OK
	naive	2048	18.710	16.646	OK
	unroll	2048	1.861	1.602	OK
sum	extreme	100000000	39.238	36.002	OK
	naive	100000000	58.782	55.612	OK
	superscalar	100000000	43.677	39.525	OK
	unroll	100000000	43.062	39.772	OK

图 6.1 和表 6.1 表明，在真正用于 profiling 的 matvec 大规模案例中，extreme 仍然保持领先。相对于 naive 的 28.302 ms，extreme 仅需 1.665 ms，达到 **17.00×** 加速；cache_opt 和 unroll 分别达到 **12.59×** 与 **11.53×**。这说明对 matvec，**第一性问题始终是访存模式**，而在其后再叠加展开和更激进内核组织才有意义。

图 6.2 则说明，当输入扩大到 10^8 量级时，extreme 在两种主要构建模式下都成为最佳实现。以 RelWithDebInfo 为例，extreme 将平均时间从 60.486 ms 降到 40.535 ms，达到 **1.49×** 加速；superscalar 和 unroll 也分别达到 **1.44×** 与 **1.33×**。这说明在真正的大输入上，更激进的归约组织仍然有稳定价值。

需要注意的是，Release 下 matvec 的 unroll 与 extreme 非常接近，分别为 1.861 ms 和 1.835 ms。这意味着当数据布局和基本循环结构都已经被优化到位后，剩余差距会更多地受到编译器代码生成细节和具体微架构行为的影响，而不再是一个简单的高层算法改写问题。

7 准备开销与稳态开销的综合分析

matvec 的转置型优化存在一个容易被忽略的问题：cache_opt、unroll 与 extreme 虽然稳态 kernel 很快，但它们包含一次性准备开销。若只比较 steady-state kernel，就会高估实际收益。因此本文采用摊销成本

$$T_{\text{amortized}} = T_{\text{kernel}} + \frac{T_{\text{setup}}}{R}$$

来衡量总效益。

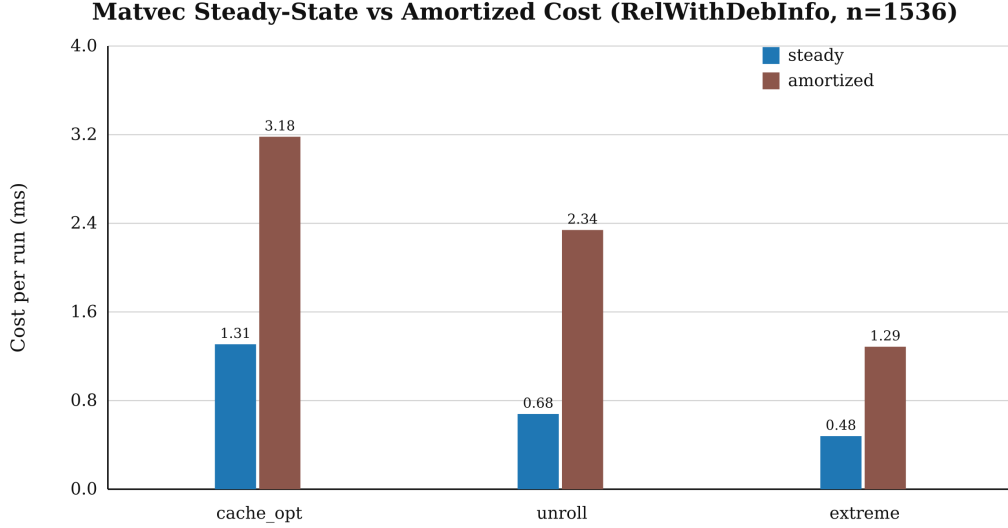


图 7.1 RelWithDebInfo 下 matvec 在 $n = 2048$ 处的稳态与摊销成本

以 RelWithDebInfo 的代表性大规模案例为例，cache_opt、unroll 与 extreme 的准备开销分别为 44.182 ms、50.674 ms 与 10.936 ms，对应重复次数均为 12 次。因此其摊销后单次成本大致为：

- cache_opt: $2.248 \text{ ms} + \frac{44.182 \text{ ms}}{12} \approx 5.930 \text{ ms}$;
- unroll: $2.455 \text{ ms} + \frac{50.674 \text{ ms}}{12} \approx 6.678 \text{ ms}$;
- extreme: $1.665 \text{ ms} + \frac{10.936 \text{ ms}}{12} \approx 2.576 \text{ ms}$ 。

图 7.1 和上述计算一起说明，extreme 不仅稳态 kernel 最快，而且准备阶段控制得也更好，因此在此“总成本”意义下仍保持明显优势。这个结论很重要，因为它表明 extreme 的优势不是只存在于“忽略准备开销时的理想化对比”。

8 VTune Hotspots 分析

8.1 采样设置与结果解释边界

本实验使用 RelWithDebInfo 二进制进行 VTune Hotspots 分析，代表性参数如下：

- matvec: naive、cache_opt、unroll、extreme，均取 $n = 2048$, repeat=50;
- sum: naive、superscalar、unroll、extreme，均取 $n = 100000000$, repeat=30。

当前采样方式是 software Hotspots 的 cpu:stack 模式，间隔为 10.000 ms。由于优化后程序本身很快，因此每次 run 的总样本数并不高，分析时更应关注热点归因是否合理以及热点是否集中到预期 kernel，而不是机械比较样本绝对数量。

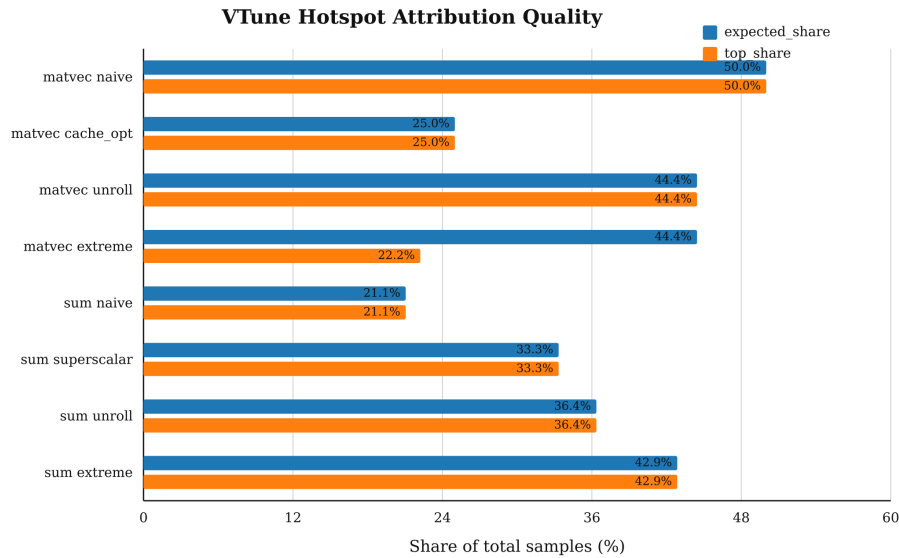


图 8.1 VTune 采样中预期核心 kernel 的样本占比

表 8.1 VTune Hotspots 采样摘要

Run	Total samples	Expected-kernel share (%)	Top hotspot
matvec naive	10.000	50.000	[Loop at line 22 in <code>matvec_naive_f64</code>]
matvec cache_opt	8.000	25.000	[Loop at line 24 in <code>matvec_cache_opt_f64</code>]
matvec unroll	9.000	44.400	[Loop at line 31 in <code>matvec_unroll4_f64</code>]
matvec extreme	9.000	44.400	[Loop@0x163610 in <code>func@0x163400</code>]
sum naive	19.000	21.100	[Loop at line 17 in <code>sum_naive_f64</code>]
sum superscalar	24.000	33.300	[Loop at line 29 in <code>sum_superscalar_f64</code>]
sum unroll	22.000	36.400	[Loop at line 27 in <code>sum_unroll4_f64</code>]
sum extreme	28.000	42.900	[Loop at line 78 in <code>sum_extreme_f64_streaming_impl</code>]

从图 8.1 和表 8.1 可以归纳出以下结论：

1. `matvecnaive` 的 hottest samples 主要落在 `matvec_naive_f64` 的内层循环上，这与“按列跨步访问导致内层循环成为瓶颈”的预期完全一致。
2. `matveccache_opt`、`matvecunroll` 与 `matvecextreme` 中，样本开始更多地落在转置或优化后的核心 kernel 上，说明驱动开销已不再主导。特别是 `matvecextreme` 中出现了 `_mm_loadu_pd` 和 `_mm_mul_pd` 等热点，说明该版本的热路径已经更贴近底层向量和寄存器操作。
3. `sumnaive` 的热点结构仍比较分散，除了核心循环外，还能看到一部分样本分布在参考实现和驱动逻辑上；这和单次运行较短、热点本身不够“胖”有关。
4. `sumsuperscalar`、`sumunroll` 与 `sumextreme` 的样本更多地集中到各自的归约循环，表明优化不仅在 benchmark 数字上更快，也确实改变了真正的热路径。

整体上，8 组 run 的 Expected-kernel share 平均约为 **37.2%**。考虑到采样间隔较粗且优化后内核持续时间较短，这已经足以支撑报告中的结构性结论：**热点位置与算法层面的性能分析是一致的。**

9 进一步优化空间讨论

从当前结果看，工程已经覆盖了作业要求以及大部分低风险、高收益优化：

- 对 **matvec** 而言，最主要的问题已经通过转置与连续访存修复，继续提升大概率需要 ISA 特定向量化、更细粒度 blocking 调参，或者更依赖具体平台的预取策略。
 - 对 **sum** 而言，优化后的瓶颈更接近大规模流式归约，继续推进往往会转化为架构相关的向量归约或 size-aware auto-tuning，而不再是简单增加几个累加器就能稳定获益。
 - 从 **Release** 下 **matvec** 中 **unroll** 与 **extreme** 已十分接近这一现象也可以看出，当前实现已经进入“剩余收益由编译器代码生成和具体微架构细节主导”的阶段。
- 因此，综合收益、复杂度和可复现性之后，当前工程是一个较合理的收束点。

10 结论

本文围绕题目要求完成了两个 CPU 架构编程实验问题的工程化实现，并得到如下结论：

1. 对 **matvec**，**数据布局与缓存局部性**是决定性因素。将按列跨步访问改写为转置后的连续访问后，可获得数量级性能提升；在此基础上结合循环展开和更激进内核组织，还能继续压缩运行时间。
2. 对 **sum**，**减少依赖链、提升 ILP** 是主要方向。多累加器与循环展开可以带来稳定收益，但随着规模增大，问题更接近带宽受限场景，因此优化间差距明显小于 **matvec**。
3. **RelWithDebInfo** 与 **Release** 的对比表明，编译器优化等级固然重要，但它无法替代算法层面的结构性改进。
4. **VTune Hotspots** 的结果与 **benchmark** 数字具有一致的解释：优化后样本更加集中于预期 kernel，本身就是性能提升确实发生在热路径上的证据。

复现实验命令

重新构建与测试

```
1 ./scripts/build.sh RelWithDebInfo
2 ./scripts/test.sh RelWithDebInfo
```

批量 benchmark

```
1 ./scripts/run_bench.sh
```

代表性 profiling case

```
1 /hw1/build/RelWithDebInfo/bench_cpu_arch \
2   --task matvec --algo extreme --n 2048 --repeat 50 \
3   --warmup 3 --dtype f64 --build-mode RelWithDebInfo
4
5 /hw1/build/RelWithDebInfo/bench_cpu_arch \
6   --task sum --algo extreme --n 100000000 --repeat 30 \
7   --warmup 3 --dtype f64 --build-mode RelWithDebInfo
```

重新生成报告图表

```
1 python3 scripts/generate_report_assets.py
```